

Prompt Abaixo

Instruções

Me ajude a transformar duas ideias do Karpathy em um prompt. O prompt será para o Claude Code, que deve ser criado em um artefato em markdown, já considerando que ele está na pasta root onde será criada a /raw e a /wiki.

Me faça perguntas a respeito antes de criar o prompt como artefato markdown. Crie o prompt INTEIRAMENTE em inglês.

O prompt deve ser para ele criar todo o setup: claude.md, estrutura de pastas, regras etc.

Contexto

Tweet Karpathy

LLM Knowledge Bases Something I'm finding very useful recently: using LLMs to build personal knowledge bases for various topics of research interest. In tLinting, a large fraction of my recent token throughput is going less into manipulating code, and more into manipulating knowledge (stored as markdown and images). The latest LLMs are quite good at it. So: Data ingest: I index source documents (articles, papers, repos, datasets, images, etc.) into a raw/ directory, then I use an LLM to incrementally "compile" a wiki, which is just a collection of .md files in a directory structure. The wiki includes summaries of all the data in raw/, backlinks, and then it categorizes data into concepts, writes articles for them, and links them all. To convert web articles into .md files I like to use the Obsidian Web Clipper extension, and then I also use a hotkey to download all the related images to local so that my LLM can easily reference them. IDE: I use Obsidian as the IDE "frontend" where I can view the raw data, the the compiled wiki, and the derived visualizations. Important to note that the LLM writes and maintains all of the data of the wiki, I rarely touch it directly. I've played with a few Obsidian plugins to render and view data in other ways (e.g. Marp for slides). Q&A: Where things get interesting is that once your wiki is big enough (e.g. mine on some recent research is ~100 articles and ~400K words), you can ask your LLM agent all kinds of complex questions against the wiki, and it will go off, research the answers, etc. I thought I had to reach for fancy RAG, but the LLM has been pretty good about auto-maintaining

 contato@dascia.com.br

 @anwar.hermuche



index files and brief summaries of all the documents and it reads all the important related data fairly easily at this ~small scale. Output: Instead of getting answers in text/terminal, I like to have it render markdown files for me, or slide shows (Marp format), or matplotlib images, all of which I then view again in Obsidian. You can imagine many other visual output formats depending on the query. Often, I end up "filing" the outputs back into the wiki to enhance it for further queries. So my own explorations and queries always "add up" in the knowledge base. Linting: I've run some LLM "health checks" over the wiki to e.g. find inconsistent data, impute missing data (with web searchers), find interesting connections for new article candidates, etc., to incrementally clean up the wiki and enhance its overall data integrity. The LLMs are quite good at suggesting further questions to ask and look into. Extra tools: I find myself developing additional tools to process the data, e.g. I vibe coded a small and naive search engine over the wiki, which I both use directly (in a web ui), but more often I want to hand it off to an LLM via CLI as a tool for larger queries. Further explorations: As the repo grows, the natural desire is to also think about synthetic data generation + finetuning to have your LLM "know" the data in its weights instead of just context windows. TLDR: raw data from a given number of sources is collected, then compiled by an LLM into a .md wiki, then operated on by various CLIs by the LLM to do Q&A and to incrementally enhance the wiki, and all of it viewable in Obsidian. You rarely ever write or edit the wiki manually, it's the domain of the LLM. I think there is room here for an incredible new product instead of a hacky collection of scripts.

LLM Wiki

LLM Wiki

A pattern for building personal knowledge bases using LLMs.

This is an idea file, it is designed to be copy pasted to your own LLM Agent (e.g. OpenAI Codex, Claude Code, OpenCode / Pi, or etc.). Its goal is to communicate the high level idea, but your agent will build out the specifics in collaboration with you.

The core idea

Most people's experience with LLMs and documents looks like RAG: you upload a collection of files, the LLM retrieves relevant chunks at query time, and generates an answer. This works, but the LLM is rediscovering knowledge from scratch on every question. There's no accumulation. Ask a subtle question that requires synthesizing



five documents, and the LLM has to find and piece together the relevant fragments every time. Nothing is built up. NotebookLM, ChatGPT file uploads, and most RAG systems work this way.

The idea here is different. Instead of just retrieving from raw documents at query time, the LLM incrementally builds and maintains a persistent wiki — a structured, interlinked collection of markdown files that sits between you and the raw sources. When you add a new source, the LLM doesn't just index it for later retrieval. It reads it, extracts the key information, and integrates it into the existing wiki — updating entity pages, revising topic summaries, noting where new data contradicts old claims, strengthening or challenging the evolving synthesis. The knowledge is compiled once and then kept current, not re-derived on every query.

This is the key difference: the wiki is a persistent, compounding artifact. The cross-references are already there. The contradictions have already been flagged. The synthesis already reflects everything you've read. The wiki keeps getting richer with every source you add and every question you ask.

You never (or rarely) write the wiki yourself — the LLM writes and maintains all of it. You're in charge of sourcing, exploration, and asking the right questions. The LLM does all the grunt work — the summarizing, cross-referencing, filing, and bookkeeping that makes a knowledge base actually useful over time. In practice, I have the LLM agent open on one side and Obsidian open on the other. The LLM makes edits based on our conversation, and I browse the results in real time — following links, checking the graph view, reading the updated pages. Obsidian is the IDE; the LLM is the programmer; the wiki is the codebase.

This can apply to a lot of different contexts. A few examples:

Personal: tracking your own goals, health, psychology, self-improvement — filing journal entries, articles, podcast notes, and building up a structured picture of yourself over time.

Research: going deep on a topic over weeks or months — reading papers, articles, reports, and incrementally building a comprehensive wiki with an evolving thesis.

Reading a book: filing each chapter as you go, building out pages for characters, themes, plot threads, and how they connect. By the end you have a rich companion wiki. Think of fan wikis like Tolkien Gateway — thousands of interlinked pages covering characters, places, events, languages, built by a community of volunteers



over years. You could build something like that personally as you read, with the LLM doing all the cross-referencing and maintenance.

Business/team: an internal wiki maintained by LLMs, fed by Slack threads, meeting transcripts, project documents, customer calls. Possibly with humans in the loop reviewing updates. The wiki stays current because the LLM does the maintenance that no one on the team wants to do.

Competitive analysis, due diligence, trip planning, course notes, hobby deep-dives — anything where you're accumulating knowledge over time and want it organized rather than scattered.

Architecture

There are three layers:

Raw sources — your curated collection of source documents. Articles, papers, images, data files. These are immutable — the LLM reads from them but never modifies them. This is your source of truth.

The wiki — a directory of LLM-generated markdown files. Summaries, entity pages, concept pages, comparisons, an overview, a synthesis. The LLM owns this layer entirely. It creates pages, updates them when new sources arrive, maintains cross-references, and keeps everything consistent. You read it; the LLM writes it.

The schema — a document (e.g. CLAUDE.md for Claude Code or AGENTS.md for Codex) that tells the LLM how the wiki is structured, what the conventions are, and what workflows to follow when ingesting sources, answering questions, or maintaining the wiki. This is the key configuration file — it's what makes the LLM a disciplined wiki maintainer rather than a generic chatbot. You and the LLM co-evolve this over time as you figure out what works for your domain.

Operations

Ingest. You drop a new source into the raw collection and tell the LLM to process it. An example flow: the LLM reads the source, discusses key takeaways with you, writes a summary page in the wiki, updates the index, updates relevant entity and concept pages across the wiki, and appends an entry to the log. A single source might touch 10-15 wiki pages. Personally I prefer to ingest sources one at a time and stay involved — I read the summaries, check the updates, and guide the LLM on what to emphasize. But you could also batch-ingest many sources at once with less



supervision. It's up to you to develop the workflow that fits your style and document it in the schema for future sessions.

Query. You ask questions against the wiki. The LLM searches for relevant pages, reads them, and synthesizes an answer with citations. Answers can take different forms depending on the question — a markdown page, a comparison table, a slide deck (Marp), a chart (matplotlib), a canvas. The important insight: good answers can be filed back into the wiki as new pages. A comparison you asked for, an analysis, a connection you discovered — these are valuable and shouldn't disappear into chat history. This way your explorations compound in the knowledge base just like ingested sources do.

Lint. Periodically, ask the LLM to health-check the wiki. Look for: contradictions between pages, stale claims that newer sources have superseded, orphan pages with no inbound links, important concepts mentioned but lacking their own page, missing cross-references, data gaps that could be filled with a web search. The LLM is good at suggesting new questions to investigate and new sources to look for. This keeps the wiki healthy as it grows.

Indexing and logging

Two special files help the LLM (and you) navigate the wiki as it grows. They serve different purposes:

index.md is content-oriented. It's a catalog of everything in the wiki — each page listed with a link, a one-line summary, and optionally metadata like date or source count. Organized by category (entities, concepts, sources, etc.). The LLM updates it on every ingest. When answering a query, the LLM reads the index first to find relevant pages, then drills into them. This works surprisingly well at moderate scale (~100 sources, ~hundreds of pages) and avoids the need for embedding-based RAG infrastructure.

log.md is chronological. It's an append-only record of what happened and when — ingests, queries, lint passes. A useful tip: if each entry starts with a consistent prefix (e.g. `## [2026-04-02] ingest | Article Title`), the log becomes parseable with simple unix tools — `grep "^## \" log.md | tail -5` gives you the last 5 entries. The log gives you a timeline of the wiki's evolution and helps the LLM understand what's been done recently.



Optional: CLI tools

At some point you may want to build small tools that help the LLM operate on the wiki more efficiently. A search engine over the wiki pages is the most obvious one — at small scale the index file is enough, but as the wiki grows you want proper search. qmd is a good option: it's a local search engine for markdown files with hybrid BM25/vector search and LLM re-ranking, all on-device. It has both a CLI (so the LLM can shell out to it) and an MCP server (so the LLM can use it as a native tool). You could also build something simpler yourself — the LLM can help you vibe-code a naive search script as the need arises.

Tips and tricks

Obsidian Web Clipper is a browser extension that converts web articles to markdown. Very useful for quickly getting sources into your raw collection.

Download images locally. In Obsidian Settings → Files and links, set "Attachment folder path" to a fixed directory (e.g. raw/assets/). Then in Settings → Hotkeys, search for "Download" to find "Download attachments for current file" and bind it to a hotkey (e.g. Ctrl+Shift+D). After clipping an article, hit the hotkey and all images get downloaded to local disk. This is optional but useful — it lets the LLM view and reference images directly instead of relying on URLs that may break. Note that LLMs can't natively read markdown with inline images in one pass — the workaround is to have the LLM read the text first, then view some or all of the referenced images separately to gain additional context. It's a bit clunky but works well enough.

Obsidian's graph view is the best way to see the shape of your wiki — what's connected to what, which pages are hubs, which are orphans.

Marp is a markdown-based slide deck format. Obsidian has a plugin for it. Useful for generating presentations directly from wiki content.

Dataview is an Obsidian plugin that runs queries over page frontmatter. If your LLM adds YAML frontmatter to wiki pages (tags, dates, source counts), Dataview can generate dynamic tables and lists.

The wiki is just a git repo of markdown files. You get version history, branching, and collaboration for free.

Why this works



The tedious part of maintaining a knowledge base is not the reading or the thinking — it's the bookkeeping. Updating cross-references, keeping summaries current, noting when new data contradicts old claims, maintaining consistency across dozens of pages. Humans abandon wikis because the maintenance burden grows faster than the value. LLMs don't get bored, don't forget to update a cross-reference, and can touch 15 files in one pass. The wiki stays maintained because the cost of maintenance is near zero.

The human's job is to curate sources, direct the analysis, ask good questions, and think about what it all means. The LLM's job is everything else.

The idea is related in spirit to Vannevar Bush's Memex (1945) — a personal, curated knowledge store with associative trails between documents. Bush's vision was closer to this than to what the web became: private, actively curated, with the connections between documents as valuable as the documents themselves. The part he couldn't solve was who does the maintenance. The LLM handles that.

Note

This document is intentionally abstract. It describes the idea, not a specific implementation. The exact directory structure, the schema conventions, the page formats, the tooling — all of that will depend on your domain, your preferences, and your LLM of choice. Everything mentioned above is optional and modular — pick what's useful, ignore what isn't. For example: your sources might be text-only, so you don't need image handling at all. Your wiki might be small enough that the index file is all you need, no search engine required. You might not care about slide decks and just want markdown pages. You might want a completely different set of output formats. The right way to use this is to share it with your LLM agent and work together to instantiate a version that fits your needs. The document's only job is to communicate the pattern. Your LLM can figure out the rest.

